

Programmation Avancée — Projet de fin de semestre n° 1

Avril 2026

Ce projet peut être réalisé en **Scheme**, ou en **Ruby** (auquel cas reportez-vous au § 2). Une réalisation en **Prolog** est également possible, auquel cas elle pourrait s'inspirer de l'exemple des cannibales et des missionnaires, vu à l'occasion des cinquième et sixième séries de travaux pratiques. Une version en **Java** peut aussi s'envisager, quoique difficilement, à mon humble avis.

1 Description

L'objectif est de réaliser un simulateur d'un mini-réseau ferroviaire où circulent plusieurs trains — au moins trois —, sur des voies distinctes deux à deux, excepté un seul point commun que nous appellerons la gare centrale. Les longueurs des voies des trains sont elles aussi distinctes deux à deux. Globalement, un tel réseau a la forme reproduite en figure 1 pour trois trains. Soit un train T , notons lg_T la longueur de son circuit. Nous considérerons que la borne n° 0 correspond à la gare centrale et ferons l'hypothèse d'événements discrets au cours desquels le train T passe successivement devant les bornes suivantes :

1 2 ... $lg_T - 1$ 0 1 ... $lg_T - 1$ 0 ...

En **Scheme**, un tel comportement peut être simulé par l'envoi du message « **incr** » au résultat de la fonction **make-top** suivante :

```
(define (make-top n)
  (let ((i 0))
    (lambda (msg)
      (cond ((eq? msg 'see) (lambda () i))      ; Voir la valeur de la borne.
            ((eq? msg 'incr) (lambda ()
                               ;; Passer à la borne suivante :
                               (set! i (modulo (+ i 1) n))
                               i))))))
```

Dans la gare centrale, les trains sont mis en attente dans une file gérée en *FIFO*¹, que vous pourrez éventuellement gérer au moyen d'une structure de *tlist*². Au départ, tous les trains sont en attente à la gare centrale. Ensuite, voici comment s'effectue le déroulement des opérations, pour chaque passage à l'état suivant :

- s'il existe un train dans la file d'attente de la gare, nous le faisons partir (mais nous n'en faisons partir qu'un) ;

1. « *First Input, First Output* » : premier entré, premier sorti.

2. Cette structure est décrite dans l'exercice 1.38 du premier chapitre de votre cours polycopié.

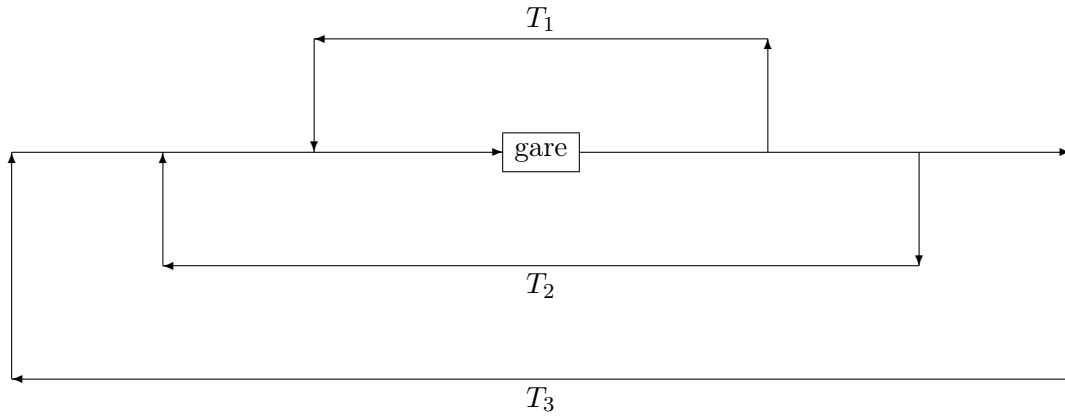


Figure 1 : Notre circuit.

-
- les autres trains passent à la borne suivante : si l'un d'eux arrive à la borne n° 0, il passe en état d'attente et rentre dans la file de la gare.

Le résultat est une liste linéaire dont chaque membre sera une liste linéaire d'au moins quatre éléments, puisqu'il vous est demandé d'inclure au moins trois trains :

- la liste des noms des trains en attente à la gare,
- la borne où se trouve le train n° 1,
- celle où se trouve le train n° 2,
- celle où se trouve le train n° 3,
- et ainsi de suite ;

le premier élément d'une telle liste étant, bien sûr :

$$((\langle train_1 \rangle \dots \langle train_n \rangle) \underbrace{0 \dots 0}_{n \text{ occurrences}})$$

avec $n \in \mathbb{N}$ et $n \geq 3$, « $\langle train_i \rangle$ » ($1 \leq i \leq n$) étant le nom du train correspondant. (Bien sûr, si un train est en attente à la gare, la valeur de la borne doit être 0.) La longueur de la liste linéaire résultat est un argument de la fonction qui la construit.

Un exemple de trace — qui utilise trois trains, **train-1**, **train-2**, **train-3**, les longueurs respectives de leurs circuits étant 10, 14 et 18 — vous est donné dans la figure 2.

La meilleure solution pour gérer les trains consiste à les dériver d'une fonction générale **make-train** dont les arguments sont le nom du train et la longueur de son circuit (*cf.* figure 2). Quant à la gare, représentée par une variable que nous appellerons **train-station**, elle utilise une clôture lexicale comportant la liste des trains en attente — rappelons qu'au départ, tous les trains sont en attente à la gare — et réagit aux messages « **see-queue** », « **queueing** » (un train entre), « **dequeueing** » (un train sort).

```

(define train-1 (make-train 'train-1 10))
(define train-2 (make-train 'train-2 14))
(define train-3 (make-train 'train-3 18))

(define example (trains (list train-1 train-2 train-3) 35))
example ⇒ (((train-1 train-2 train-3) 0 0 0)    ; Au départ, les trois trains sont dans la gare.
            ((train-2 train-3) 1 0 0)          ; train-1 est parti.
            ((train-3) 2 1 0)                  ; train-2 aussi.
            (() 3 2 1)                          ; Les trois trains sont partis.
            (() 4 3 2)
            (() 5 4 3)
            (() 6 5 4)
            (() 7 6 5)
            (() 8 7 6)
            (() 9 8 7)
            ((train-1) 0 9 8)                    ; train-1 est revenu à la gare...
            (() 1 10 9)                          ; ... et est reparti.
            (() 2 11 10)
            (() 3 12 11)
            (() 4 13 12)
            ((train-2) 5 0 13)
            (() 6 1 14)
            (() 7 2 15)
            (() 8 3 16)
            (() 9 4 17)
            ((train-1 train-3) 0 5 0)            ; train-1 et train-3 sont arrivés ensemble à la gare...
            ((train-3) 1 6 0)                    ; ... mais seul train-1 est parti.
            (() 2 7 1)                          ; Au tour de train-3.
            (() 3 8 2)
            (() 4 9 3)
            (() 5 10 4)
            (() 6 11 5)
            (() 7 12 6)
            (() 8 13 7)
            ((train-2) 9 0 8)
            ((train-1) 0 1 9)
            (() 1 2 10)
            (() 2 3 11)
            (() 3 4 12)
            (() 4 5 13))

```

Figure 2: Exemple de simulation de circulation de trains.

2 Version en Ruby

Dans cette version, les trains — dérivés d’une fonction de création de trains — peuvent être représentés par des *threads* ou des *fibres*, mais ce n’est pas une obligation. Rappelons que nous considérons des événements *discrets* : le résultat à fournir doit montrer clairement l’évolution du système global lors du passage d’un état à l’état suivant. En outre, prêter attention — si vous choisissez cette possibilité — à ce que votre programme en Ruby soit opérationnel avec la version installée dans les salles de travaux pratiques.